UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

IF 451
Advance Web Programming

Meet4 - Mini Project: Student Management (JSON)

Wirawan Istiono, S.Kom., M.Kom

Meet 4

Objective:

Students are able to implement various types of libraries and database to create web application using Node JS (C3)

OUTLINE

- *Create a Node.js CLI or server app to manage student data.*
- *Implement add, list, update, delete features.*

Create a Node.js CLI or server app to manage student data.

What is Node.js CLI?

CLI = Command Line Interface

An application that runs through the terminal/command prompt, not through a browser.

We interact by typing commands, and the application outputs them in the terminal.

Simple examples of CLI: npm install, git status, or node myscript.js.

So, a Node.js CLI app means we create a program with Node.js that runs in the terminal, not a web server.

Node.js CLI example - Show

1. First, create new file JSON, with name "students.json"
2. Write the JSON code below
3. Create new file with name "app.js" and write the code beside, to call json file and show all data in terminal
4. Run app.js with command "node app.js"

```json
[
  {
    "nama": "Andi",
    "umur": 20,
    "hobi": ["membaca","coding"]
  },
  {
    "nama": "Budi",
    "umur": 22,
    "hobi": ["gaming","olahraga"]
  }
]
```

```js
const fs = require("fs");

let rawData = fs.readFileSync("students.json");
let students = JSON.parse(rawData);

console.log("Daftar Siswa:");
students.forEach((elm, i) => {
    console.log("Nomor: " + i+1);
    console.log("Nama: " + elm.nama);
    console.log("Umur: " + elm.umur);
    console.log("Hobi: " + elm.hobi[0] + " dan " + elm.hobi[1]);
    console.log("==========================");
});
```

Process Global Object NodeJS

Is it possible to create CRUD (Create, Read, Update and Delete) in CLI?

Before you learn CRUD in CLI, you need to know, that NodeJS has Global Object to access information about the Node.js that is running (runtime process). To access that information, the function is “process”

With processes, we can:

- Get arguments from the command line (process.argv)
- Check the environment (process.env)
- Exit the program (process.exit())

Process Global Object NodeJS example

Sample "process" code in node.js

process.argv[2]

1. **process** is code to access runtime global object information node.js
2. **argv (argument value)** is an array containing a list of arguments when you run Node.js from the terminal.

Example.

Write this single code in app.js

console.log(process.argv);

Type this command in your terminal,

node app.js hello world

```
meet4> node app.js Hello World
```

```
[
  'C:\\nvm4w\\nodejs\\node.exe',
  'E:\\UMN\\IF451 - Advance Web\\Modul\\Lecture\\sample_project\\meet4\\app.js',
  'Hello',
  'World'
]
```

Sample command with process.argv

1. Write code in “app.js”
2. Run in command prompt like the sample below

```
const perintahCMD = process.argv;
if(perintahCMD[2]=="tambah") {
    console.log("Data akan ditambah");
} else {
    console.log("Data akan dihapus");
}
```

```
PS E:\UMN\IF451 - Advance Web\Modul\Lecture\sample_project\meet4> node app.js tambah
Data akan ditambah
PS E:\UMN\IF451 - Advance Web\Modul\Lecture\sample_project\meet4> node app.js ngapainya
Data akan dihapus
```

Conclusion:

We can get attribute after in command prompt to use in nodejs code to create simple CLI application

Implement add, list, update, delete features. (CLI)

Simple CLI app (Show data – **R**ead)

```
const fs = require("fs");

let rawData = fs.readFileSync("students.json");
let students = JSON.parse(rawData);

console.log("Tuliskan 'node app.js lihat/tambah/update/hapus' (pilih salah satu)");

const perintahCMD = process.argv;
if(perintahCMD[2]=="lihat") {
    console.log("Daftar Siswa:");
    students.forEach((elm, i) => {
        console.log("Nomor: " + i+1);
        console.log("Nama: " + elm.nama);
        console.log("Umur: " + elm.umur);
        console.log("Hobi: " + elm.hobi[0] + " dan " + elm.hobi[1]);
        console.log("==========================");
    });
} else if (perintahCMD[2]=="tambah") {
    console.log("Data akan ditambah");
} else if (perintahCMD[2]=="update") {
    console.log("Data akan ditambah");
} else if (perintahCMD[2]=="hapus") {
    console.log("Data akan dihapus");
}
```

```
PS E:\UMN\IF451 - Advance Web\Modul\Lecture\sample_project\meet4> node app.js
Tuliskan 'node app.js lihat/tambah/update/hapus' (pilih salah satu)
PS E:\UMN\IF451 - Advance Web\Modul\Lecture\sample_project\meet4> node app.js lihat
Tuliskan 'node app.js lihat/tambah/update/hapus' (pilih salah satu)
Daftar Siswa:
Nomor: 01
Nama: Andi
Umur: 20
Hobi: membaca dan coding
==========================
Nomor: 11
Nama: Budi
Umur: 22
Hobi: gaming dan olahraga
==========================
```

Simple CLI app (Delete data – **D**elete)

```
const perintahCMD = process.argv;
if(perintahCMD[2]=="lihat") {
    console.log("Daftar Siswa:");
    students.forEach((elm, i) => {
        console.log("Nomor: " + i+1);
        console.log("Nama: " + elm.nama);
        console.log("Umur: " + elm.umur);
        console.log("Hobi: " + elm.hobi[0] + " dan " + elm.hobi[1]);
        console.log("==========================");
    });
} else if (perintahCMD[2]=="tambah") {
    console.log("Data akan ditambah");
} else if (perintahCMD[2]=="update") {
    console.log("Data akan ditambah");
} else if (perintahCMD[2]=="hapus") {
    console.log("Data akan dihapus");
    students = students.filter(item => item.nama !== perintahCMD[3]);
    fs.writeFileSync("students.json", JSON.stringify(students, null, 2));
}
```

Sample Command:

node app.js hapus Budi

```
PS E:\UMN\IF451 - Advance Web\Modul\Lecture\sample_project\meet4> node app.js hapus Budi
Tuliskan 'node app.js lihat/tambah/update/hapus' (pilih salah satu)
Data akan dihapus
```

Implement add, list, update, delete features.
(Server App / Website)

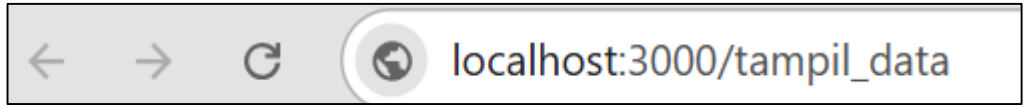
http Module NodeJS

In server app or website, there is a parameter or variable in URL like the sample beside

To get URL address and parameter or variable, you can add .method or .url attribute from req parameter, like the sample beside.

1. Write this scipt in your app.js files
2. Run your nodeJS via cmd, "node app.js"
3. Type or edit or URL browser, like this sample "localhost:3000/tampil_data"
4. See the result in your terminal

```
const http = require("http");

const server = http.createServer((req, res)=>{
  console.log("Req Method: " + req.method);
  console.log("Req URL: " + req.url);
});

server.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```




```
Server running at http://localhost:3000
Req Method: GET
Req URL: /
Req Method: GET
Req URL: /tampil_data
```

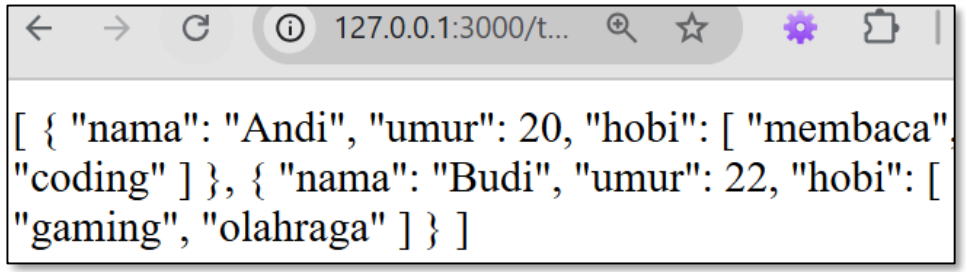
Server app (**R**ead) JSON files

```
const http = require("http");
const fs = require("fs");

const server = http.createServer((req, res)=>{
    const raw = fs.readFileSync("students.json");
    const data = JSON.parse(raw);

    res.writeHead(200, {"Content-Type":"text/html"});
    if(req.method=="GET") {
        if(req.url=="/tampil_data") {
            res.write(JSON.stringify(data, null, 2));
        }
    }
    res.end();
});

server.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

Beside is the sample to get data JSON, when user type

"http://localhost:3000/tampil_data"

In the browser url address.



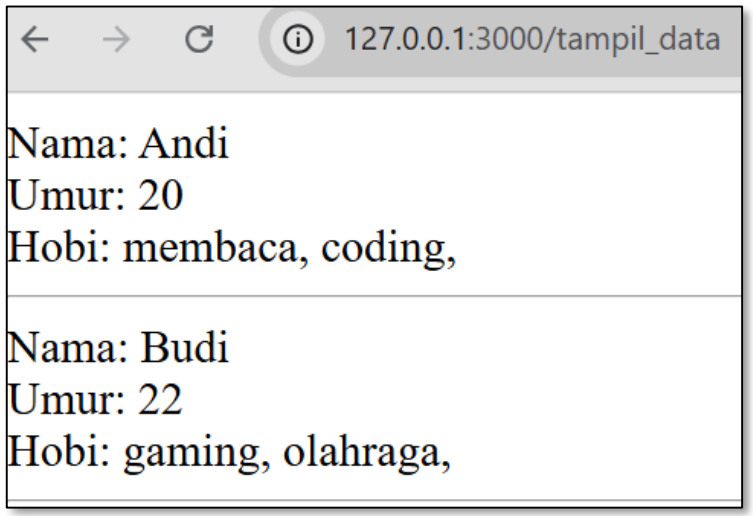
Server app (**R**ead) JSON detail parameter

```
const http = require("http");
const fs = require("fs");

const server = http.createServer((req, res)=>{
    const raw = fs.readFileSync("students.json");
    const data = JSON.parse(raw);
    res.setHeader('Content-Type', 'text/html');

    if(req.url=="/tampil_data") {
      data.forEach(element => {
        console.log(element);
        res.write("Nama: " + element.nama + "<br>");
        res.write("Umur: " + element.umur + "<br>");
        res.write("Hobi: ");
        for(let a=0; a<element.hobi.length; a++) {
          res.write(element.hobi[a] + ", ");
        }
        res.write("<hr>");
      });
    }
    res.end();
});
```

Beside is the sample to get data JSON with detail parameter, when user type

"http://localhost:3000/tampil_data"

In the browser url address.



Multiple parameter JSON files (1/2)

1. To get multiple parameter (“edit”, “Andi” and “21”), you can write sample code beside.
2. You will get the result, like sample beside.

```
const http = require("http");

const server = http.createServer((req, res)=>{
    console.log("URL: " + req.url);
});

server.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

```
Server running at http://localhost:3000
URL: /edit/Andi/21
```

Multiple parameter JSON files (2/2)


```
Server running at http://localhost:3000
URL: /edit/Andi/21
```

3. After get the result "/edit/Andi/21"
4. To split every parameter, you can use "split" function from array. To use "split", write the code like the sample beside.

```
Server running at http://localhost:3000
URL: /edit/Andi/21
parameter1:edit
parameter2:Andi
parameter3:21
```

```
const http = require("http");

const server = http.createServer((req, res)=>{
    console.log("URL: " + req.url);
    const arrURL = req.url.split("/");
    console.log("parameter1:" + arrURL[1]);
    console.log("parameter2:" + arrURL[2]);
    console.log("parameter3:" + arrURL[3]);
});

server.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

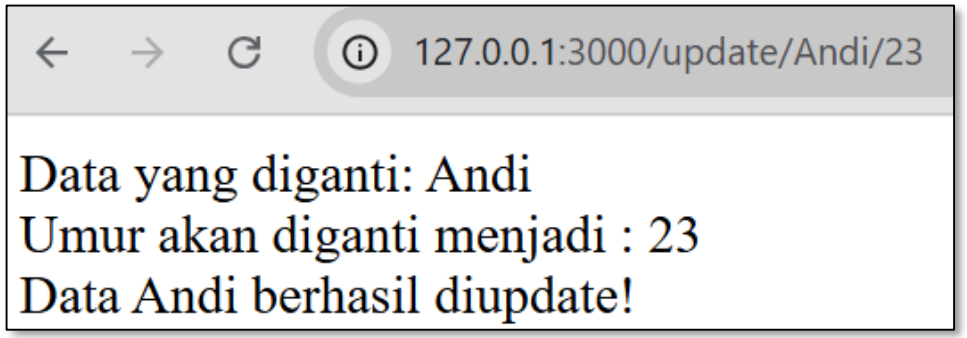
Server app (**U**pdate) JSON files

```
        res.write("<hr>");
      });
    } else if (arrURL[1]=="update") {
      const keyDiganti = arrURL[2];
      res.write("Data yang diganti: " + keyDiganti);
      res.write("<br>Umur akan diganti menjadi : " + arrURL[3]);

      const index = data.findIndex(item => item.nama === keyDiganti);
      if (index !== -1) {
        data[index].umur = arrURL[3];

        fs.writeFileSync("students.json", JSON.stringify(data, null, 2));
        res.write("<br>Data "+keyDiganti+" berhasil diupdate!");
      } else {
        res.write("<br>Data "+keyDiganti+" tidak ditemukan.");
      }
    }
    res.end();
});
```

Beside is the sample to udpate data JSON, when url has this parameter

"http://127.0.0.1:3000/update/Andi/23"

In the browser url address.



Advantages and disadvantages of Node.js CLI

Advantages:

- Simple & lightweight → No server required, just run in the terminal.
- Fast for automating tasks → suitable for scripts, such as processing JSON files, generating reports, migrating databases, etc.
- No internet connection required → everything runs locally.
- Quick to build → just use process.argv or a CLI library (e.g., commander or yargs).
- Easy to integrate into DevOps, cron jobs, or build tools.

Disadvantages:

- No visual UI → everything is text-based in the terminal, not very user-friendly.
- Not suitable for multiple users → usually only used by one developer/operator.
- Must be run manually → unless installed in cron/scheduler.
- Not easily accessible from a browser or other application (needs to be invoked via script/command).

Advantages and disadvantages of Node.js Server app

Advantages:

- Multi-user capability → can be accessed by many people simultaneously via a browser.
- User-friendly → can be provided with a UI (HTML/CSS/React/Vue) or API (REST/GraphQL).
- Always ready to accept requests as long as the server is up.
- Extensive integration → can be used for web apps, mobile apps, IoT, etc.
- More flexible → supports authentication, large databases, and various types of services.

Disadvantages:

- More complex → requires more setup (Express.js, middleware, routing, etc.).
- Requires more resources → runs continuously in the background, requires server monitoring.
- Must be deployed → to be accessible to multiple users (e.g., to a VPS, cloud, or hosting).
- Requires extra security → because the server is vulnerable to attacks (SQL injection, XSS, etc.).

Assignment

Assignment

1. Create the CLI application with nodejs, where the application can do CRUD (create, read, update and delete data) to json file.
2. You should give clear and specific instruction to the user, so user can success to do CRUD in your CLI application

Submit your assignment to elearning with this following rules:

1. Save **all you code js and json** into one folder.
2. Zip your code with name "nim_name_if451_meet4.zip"

NEXT WEEK'S OUTLINE

- Introduction to MySQL Database

REFERENCES

- B. Lawson and J. Encinas, Node.js Web Development, 6th ed., Packt Publishing, 2023
- R. Raj, Learning Node.js Development, Packt Publishing, 2018.
- E. Cantelon, M. Harter, T. Holowaychuk, and N. Rajlich, Node.js in Action, 2nd ed., Manning Publications, 2017.
- Maulana, a., bau, r.t.r., munawar, z., setiawan, r., aisa, s., istiono, w., irfan, a., pratama, y.a., ramadhany, e.d., pomalingo, s. And permana, a.a., 2023. *Pemrograman web 101: memahami dasar-dasar untuk mengembangkan situs web*. Get press indonesia.
- The Node.js Foundation, "Node.js Documentation," [Online]. Available: https://nodejs.org/en/docs/. [Accessed: Jun. 5, 2025].
- Express.js Team, "Express.js Documentation," [Online]. Available: https://expressjs.com/. [Accessed: Jun. 5, 2025].

Visi

Menjadi Program Studi Strata Satu Informatika **unggulan** yang menghasilkan lulusan **berwawasan internasional** yang **kompeten** di bidang Ilmu Komputer (*Computer Science*), **berjiwa wirausaha** dan **berbudi pekerti luhur**.

INFORMATIKA
UMN
UNIVERSITAS MULTIMEDIA NUSANTARA

Misi

1. Menyelenggarakan pembelajaran dengan teknologi dan kurikulum terbaik serta didukung tenaga pengajar profesional.
2. Melaksanakan kegiatan penelitian di bidang Informatika untuk memajukan ilmu dan teknologi Informatika.
3. Melaksanakan kegiatan pengabdian kepada masyarakat berbasis ilmu dan teknologi Informatika dalam rangka mengamalkan ilmu dan teknologi Informatika.